

**UNIVERSITY OF KARACHI**  
Department of Computer Science  
Software Project Management — 506 (LAB 3+0)

# CONGINE

*Deterministic Output Governance for AI-Assisted Software Development*

## PROJECT CHARTER & BUSINESS CASE

Business Justification, Project Charter and Management Plan

| Field              | Detail                                                                                                                              |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Document Reference | CGN-DOC-03                                                                                                                          |
| Project Name       | Congine — Deterministic Output Governance Engine                                                                                    |
| Course             | Software Project Management — 506 (LAB 3+0)                                                                                         |
| Instructor         | Dr. Humera                                                                                                                          |
| Semester           | 6th                                                                                                                                 |
| Assignment         | Assignment-01: Project Charter Development                                                                                          |
| Team Members       | 1. _____ Roll No: _____<br>2. _____ Roll No: _____<br>3. _____ Roll No: _____<br>4. _____ Roll No: _____<br>5. _____ Roll No: _____ |
| Version            | 1.0                                                                                                                                 |
| Date of Submission | _____                                                                                                                               |
| Status             | Submitted for Evaluation                                                                                                            |

## Table of Contents

---

1. Project Charter
2. Business Case
3. Market and User Analysis
4. Product Requirements — Business Perspective
5. Success Criteria and Measurement
6. Risk Management
7. Work Breakdown Structure
8. Project Schedule and Milestones
9. Team Structure and Responsibilities
10. Communication Management Plan
11. Quality Management Plan
12. Change and Scope Management
13. Exclusions and Deferred Scope
14. Conclusion

# 1. Project Charter

## Purpose of This Section

This section constitutes the formal Project Charter as required by Assignment-01 of the Software Project Management laboratory. It authorizes the project, defines its boundaries, identifies its stakeholders, enumerates its deliverables and establishes its timeline. Sections 2 onward provide the supporting business and management detail from which the charter is derived.

## 1.1 Project Title and Identification

| Field                     | Detail                                                                                |
|---------------------------|---------------------------------------------------------------------------------------|
| <b>Project Title</b>      | Congine — Deterministic Output Governance Engine for AI-Assisted Software Development |
| <b>Short Name</b>         | Congine                                                                               |
| <b>Project Identifier</b> | CGN-2026                                                                              |
| <b>Project Type</b>       | Software product development (developer tooling / software development kit)           |
| <b>Sponsoring Course</b>  | Software Project Management — 506 (LAB 3+0)                                           |
| <b>Charter Version</b>    | 1.0                                                                                   |
| <b>Charter Date</b>       | _____                                                                                 |
| <b>Prepared By</b>        | Congine Project Team                                                                  |

## 1.2 Project Initiation and Business Case Summary

### 1.2.1 Background

Artificial intelligence coding assistants have been adopted rapidly across professional software engineering teams. These tools generate source code, structured data and configuration at a rate that substantially exceeds human production, and their adoption has produced measurable gains in development velocity.

The mechanisms by which organizations assure the quality of software, however, were designed for human rates of production. Code review, the principal control in most organizations, is a manual process performed by engineers with finite attention. When the volume of generated output increases substantially while review capacity remains constant, the proportion of output receiving genuine scrutiny declines.

The consequence is a governance gap. AI assistants produce output that executes correctly but does not necessarily conform to the specific architectural rules, data contracts, naming conventions or policy requirements of the organization consuming it. Such deviations are individually minor and collectively significant: accumulated over time they produce architectural drift, latent defects and audit exposure.

### 1.2.2 Reason for Undertaking the Project

No widely adopted mechanism presently exists that enforces an organization's own standards on AI-generated output automatically, rapidly and deterministically at the point of generation. Existing tools in adjacent categories either observe without enforcing, address general-purpose content rather than engineering standards, or apply probabilistic evaluation that cannot serve as the basis of organizational policy.

The project undertakes to build that mechanism: a governance layer in which requirements are expressed as machine-readable contracts authored by human engineers, and in which every relevant output is evaluated against the applicable contract by rule-based logic producing an identical verdict for identical inputs.

### 1.2.3 Expected Benefits

- **Assurance.** Organizational standards are enforced automatically and uniformly rather than depending on the availability and attention of a human reviewer.
- **Efficiency.** Non-conforming output is identified immediately and precisely, reducing the number of corrective iterations required and the associated expenditure on AI interaction.
- **Continuity.** Project decisions and validation history are preserved in a durable record, reducing the cost of resuming work, transferring work and onboarding new team members.
- **Consistency.** Work produced through different AI assistants is held to one common standard, reducing divergence within a shared codebase.
- **Auditability.** Every evaluation is recorded, producing evidence that policy was enforced rather than merely stated.

## 1.3 Project Objectives

Objectives are stated in accordance with the SMART criteria — specific, measurable, achievable, relevant and time-bound.

| ID    | Objective                                                                                                | Measure of Achievement                                                                                                       | Target Period |
|-------|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------|
| OBJ-1 | Deliver a validation engine producing deterministic verdicts against human-authored contracts            | Identical inputs produce identical verdicts across repeated executions, demonstrated by automated test                       | Phase 1       |
| OBJ-2 | Ensure validation operates within a bounded latency allowance without destabilizing the host application | Validation completes within the configured allowance; excess load is rejected in a controlled manner                         | Phase 1       |
| OBJ-3 | Establish a comprehensive automated test suite covering ordinary and adversarial conditions              | All components on the validation path covered by unit and adversarial tests; documented limits exercised at their boundaries | Phase 1       |
| OBJ-4 | Produce complete technical and business documentation for the system                                     | Concept brief, technical specification and management plan reviewed and approved                                             | Phase 1       |
| OBJ-5 | Enable AI coding assistants to consult the validation engine during their operation                      | An assistant successfully obtains a structured verdict and applies a correction based upon it                                | Phase 2       |
| OBJ-6 | Preserve validation history durably across process restarts                                              | Recorded events remain retrievable following restart, demonstrated by test                                                   | Phase 3       |
| OBJ-7 | Demonstrate consistent evaluation of output produced by more than one AI assistant                       | Outputs from differing assistants evaluated against one common contract with consistent results                              | Phase 4       |

## 1.4 Project Scope

### 1.4.1 Included in Scope

- The core validation engine and its rule set.
- The contract format and the mechanism by which contracts are loaded and cached.
- Supporting infrastructure required for safe operation within a host application: bounded execution, resilience management, controlled failure and observability.
- Integration mechanisms permitting adoption within an application and consultation by an AI coding assistant.
- A durable record of validation events.
- Security measures appropriate to a component that processes untrusted input.
- Complete technical and project documentation.

#### 1.4.2 Excluded from Scope

- A graphical user interface or web application of any kind.
- The centralized administrative service through which contracts may be distributed across an organization.
- Commercial packaging, pricing, licensing and distribution arrangements.
- Training or fine-tuning of machine learning models.
- Automatic generation of contracts by inference from an existing codebase.
- Integration with any specific proprietary development environment beyond documented general interfaces.
- Formal regulatory certification of any kind.

### 1.5 Project Stakeholders

| Stakeholder                              | Category                  | Interest in the Project                                                                          | Influence |
|------------------------------------------|---------------------------|--------------------------------------------------------------------------------------------------|-----------|
| <b>Project team</b>                      | Internal — delivery       | Successful delivery; demonstration of engineering and management competence; academic assessment | High      |
| <b>Course instructor (Dr. Humera)</b>    | External — governance     | Correct application of project management practice; quality of documentation and presentation    | High      |
| <b>Department of Computer Science</b>    | External — institutional  | Academic standards; suitability of the project as coursework                                     | Medium    |
| <b>Software development teams</b>        | External — end user       | A tool that improves output quality without impeding development velocity                        | High      |
| <b>Technical leads and architects</b>    | External — end user       | Preservation of architectural integrity as team and codebase grow                                | High      |
| <b>Engineering managers</b>              | External — decision maker | Consistency of output; predictable review burden; controlled cost                                | Medium    |
| <b>Compliance and security functions</b> | External — assurance      | Demonstrable evidence that policy is enforced; auditable records                                 | Medium    |
| <b>Peer student teams</b>                | External — collaborative  | Shared learning; reference material for comparable undertakings                                  | Low       |
| <b>AI assistant providers</b>            | External — ecosystem      | Interoperability with their tools; adherence to published interfaces                             | Low       |

## 1.6 Project Deliverables

| ID   | Deliverable                         | Description                                                         | Acceptance Criterion                                                       |
|------|-------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------|
| D-01 | Project Concept Brief               | Conceptual overview of the problem domain and solution approach     | Reviewed and approved; problem and solution clearly articulated            |
| D-02 | Technical Specification             | Requirements specification and architectural design                 | All requirements identified and traceable to architecture and verification |
| D-03 | Project Charter and Management Plan | This document: charter, business case, risks, schedule and plan     | Contains all charter elements; approved by stakeholders                    |
| D-04 | Core validation engine              | Implemented rule-based engine producing deterministic verdicts      | All functional requirements for validation satisfied and tested            |
| D-05 | Supporting infrastructure           | Bounded execution, caching, resilience, recording and configuration | Non-functional requirements demonstrably satisfied                         |
| D-06 | Automated test suite                | Unit, integration, adversarial and concurrency tests                | Comprehensive coverage of the validation path; all tests passing           |
| D-07 | Integration mechanisms              | Means by which the engine is adopted and consulted                  | A representative integration demonstrated end to end                       |
| D-08 | Contract authoring guidance         | Documentation enabling correct authorship of contracts              | A newcomer can author a correctly enforced contract using it alone         |
| D-09 | Security assessment                 | Threat model and record of defensive measures                       | All identified trust boundaries assessed and addressed                     |
| D-10 | Project presentation                | Presentation of the charter and project to the class                | Delivered within the allotted time; charter elements communicated          |

## 1.7 Assumptions and Constraints

### 1.7.1 Assumptions

| ID   | Assumption                                                                                            |
|------|-------------------------------------------------------------------------------------------------------|
| A-01 | Organizations possess engineering standards that can be expressed as machine-readable rules.          |
| A-02 | Engineering teams are willing to author and maintain contract definitions.                            |
| A-03 | AI coding assistants will consult an available advisory service when instructed to do so.             |
| A-04 | Team members remain available for the duration of the project at the anticipated level of commitment. |
| A-05 | Required development tooling and computing resources remain accessible throughout.                    |
| A-06 | A latency allowance in the low hundreds of milliseconds is acceptable to the intended users.          |

### 1.7.2 Constraints

| ID   | Constraint                                                        | Effect on the Project                                                                        |
|------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| C-01 | Academic calendar and submission deadlines                        | Scope must be sequenced so that a coherent increment is deliverable at each assessment point |
| C-02 | Part-time availability of team members alongside other coursework | Estimates must reflect reduced effective capacity; parallel work must be limited             |
| C-03 | Small team size                                                   | Roles are shared rather than specialized; documentation is essential to continuity           |

| ID          | Constraint                                                    | Effect on the Project                                                                                                      |
|-------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>C-04</b> | Determinism is non-negotiable                                 | No probabilistic component may participate in the validation decision, foreclosing several otherwise attractive approaches |
| <b>C-05</b> | The system operates within a host process it does not control | Resource consumption must be bounded and failures must not propagate                                                       |
| <b>C-06</b> | Offline operation must be supported                           | Every external dependency requires a local fallback                                                                        |
| <b>C-07</b> | No budget for commercial tooling or infrastructure            | Only freely available tooling and services may be used                                                                     |

## 1.8 Preliminary Schedule and Resource Estimate

### 1.8.1 High-Level Schedule

| Phase             | Description                                                    | Indicative Duration | Principal Deliverables |
|-------------------|----------------------------------------------------------------|---------------------|------------------------|
| <b>Initiation</b> | Charter, requirements definition, architectural design         | 2 weeks             | D-01, D-02, D-03       |
| <b>Phase 1</b>    | Core validation engine, supporting infrastructure, test suite  | 5 weeks             | D-04, D-05, D-06       |
| <b>Phase 2</b>    | Integration mechanisms and consultative service interface      | 4 weeks             | D-07                   |
| <b>Phase 3</b>    | Durable validation history and contextual recall               | 4 weeks             | Extension of D-05      |
| <b>Phase 4</b>    | Multi-assistant consistency capability                         | 4 weeks             | Extension of D-07      |
| <b>Closure</b>    | Documentation consolidation, security assessment, presentation | 2 weeks             | D-08, D-09, D-10       |

Durations are indicative and assume part-time availability. Phases 1 and 2 constitute the critical path; Phases 3 and 4 depend upon the completion of Phase 1 and, in the case of Phase 4, upon the accumulation of data produced by Phase 3.

### 1.8.2 Resource Estimate

| Resource Category              | Requirement                                                     | Estimated Cost                                   |
|--------------------------------|-----------------------------------------------------------------|--------------------------------------------------|
| <b>Human resources</b>         | Project team members, part-time throughout the project duration | Nil — academic project                           |
| <b>Development hardware</b>    | Personal computers owned by team members                        | Nil — existing resources                         |
| <b>Development software</b>    | Open-source runtime, libraries, editor and version control      | Nil — freely available                           |
| <b>Version control hosting</b> | Repository hosting for source and documentation                 | Nil — free tier                                  |
| <b>AI assistant access</b>     | Access to at least one coding assistant for integration testing | Nil to nominal — free or existing access         |
| <b>Total estimated budget</b>  | —                                                               | <b>Nil (no external expenditure anticipated)</b> |

## 1.9 Project Manager and Authority

The project manager is designated below and is authorized to allocate work among team members, determine the sequencing of activities, accept or reject deliverables against their stated acceptance criteria, and approve changes to scope within the limits described in Section 12.

Changes that alter the project objectives stated in Section 1.3, or that affect a submission deadline, require the agreement of the full team and the acknowledgement of the course instructor.

| Role                      | Name                           | Authority                                                                           |
|---------------------------|--------------------------------|-------------------------------------------------------------------------------------|
| <b>Project Manager</b>    | _____                          | Work allocation; sequencing; deliverable acceptance; scope changes within tolerance |
| <b>Technical Lead</b>     | _____                          | Architectural decisions; technical standards; design review                         |
| <b>Quality Lead</b>       | _____                          | Test strategy; acceptance verification; defect triage                               |
| <b>Documentation Lead</b> | _____                          | Documentation standards; consistency; version control of documents                  |
| <b>Project Sponsor</b>    | Dr. Humera (Course Instructor) | Overall approval; assessment; final acceptance                                      |

## 1.10 Charter Approval

By signing below, the parties acknowledge that they have reviewed this charter, agree to its contents, and authorize the commencement of the project as described.

| Role              | Name       | Signature | Date  |
|-------------------|------------|-----------|-------|
| Project Manager   | _____      | _____     | _____ |
| Team Member       | _____      | _____     | _____ |
| Team Member       | _____      | _____     | _____ |
| Team Member       | _____      | _____     | _____ |
| Team Member       | _____      | _____     | _____ |
| Course Instructor | Dr. Humera | _____     | _____ |



## 2. Business Case

### 2.1 The Opportunity

The adoption of AI coding assistants within professional engineering teams has been rapid and appears durable. This adoption has created a category of requirement that did not previously exist: the need to govern machine-generated output at the rate at which it is produced.

The requirement is not adequately met by existing categories of tooling. Static analysis tools evaluate code against general-purpose rules but not against an organization's specific architectural intent. Testing frameworks verify behaviour but not conformance to structural policy. Observability platforms record what occurred but do not prevent it. Code review, the principal existing control, does not scale to the volume now being produced.

The opportunity is therefore to establish a governance layer positioned specifically at the point where machine-generated output enters a human-governed codebase.

### 2.2 Problem Cost Analysis

The cost of the unaddressed problem falls into four categories.

| Cost Category                   | Nature of the Cost                                                                                            | Party Bearing It                                 |
|---------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| <b>Corrective effort</b>        | Developer time expended on repeated iteration with an assistant that does not know its output is unacceptable | Engineering team                                 |
| <b>Interaction expenditure</b>  | Direct charges for repeated and largely redundant AI interaction                                              | Organization                                     |
| <b>Remediation of drift</b>     | Effort required to restore architectural coherence once deviation has accumulated                             | Engineering team; disproportionately borne later |
| <b>Knowledge reconstruction</b> | Time expended re-establishing project context at every session, handover and onboarding                       | Engineering team                                 |
| <b>Assurance exposure</b>       | Risk arising from policy that is stated but not demonstrably enforced                                         | Organization; compliance function                |

The fourth and fifth categories merit particular attention because they are typically unmeasured. Knowledge reconstruction is paid in small increments that are individually unremarkable and collectively substantial. Assurance exposure produces no cost at all until the occasion on which it produces a very large one.

### 2.3 Proposed Solution

The project proposes a governance engine embedded within the development workflow, in which organizational requirements are expressed as machine-readable contracts and every relevant output is evaluated against the applicable contract by deterministic rule-based logic.

| Cost Category                  | How the Solution Addresses It                                                                                            |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Corrective effort</b>       | Non-conformance is identified immediately and precisely, permitting targeted correction rather than undirected iteration |
| <b>Interaction expenditure</b> | Fewer corrective cycles are required, and each carries less transmitted context                                          |
| <b>Remediation of drift</b>    | Deviation is prevented at the point of generation rather than remedied after accumulation                                |

| Cost Category                   | How the Solution Addresses It                                                 |
|---------------------------------|-------------------------------------------------------------------------------|
| <b>Knowledge reconstruction</b> | Validation history and decisions are preserved in a durable, queryable record |
| <b>Assurance exposure</b>       | Every evaluation is recorded, producing evidence that policy was enforced     |

## 2.4 Options Considered

Four options were assessed before the proposed approach was selected.

| Option                                    | Description                                            | Assessment                                                                                                       |
|-------------------------------------------|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Do nothing</b>                         | Continue to rely on human review as the sole control   | Rejected. Does not scale with output volume; the problem worsens as adoption increases.                          |
| <b>Strengthen review process</b>          | Add review stages and checklists                       | Rejected. Increases the burden on the constrained resource; remains inconsistent and slow.                       |
| <b>AI-based review</b>                    | Employ a second AI system to evaluate the first        | Rejected. Inherits probabilistic uncertainty; verdicts are not reproducible or auditable; adds latency and cost. |
| <b>Deterministic contract enforcement</b> | Rule-based evaluation against human-authored contracts | Selected.** Reproducible, auditable, fast, and compatible with organizational policy.                            |

### Rationale for the Selected Option

The third option merits particular comment because it is the most intuitively appealing and was given serious consideration. It was rejected on the ground that a governance control whose verdict varies between identical inputs cannot serve as the basis of an organizational policy, cannot be defended in an audit, and reproduces within the control the same uncertainty that made the control necessary. Determinism is consequently treated as a defining constraint of the project rather than as one design consideration among several.

## 2.5 Benefit Summary

| Benefit                                     | Category                   | Realized From          |
|---------------------------------------------|----------------------------|------------------------|
| Reduced corrective iteration                | Quantifiable               | Phase 1 onward         |
| Reduced AI interaction expenditure          | Quantifiable               | Phase 2 onward         |
| Prevention of architectural drift           | Quantifiable over time     | Phase 2 onward         |
| Faster resumption of work and onboarding    | Partly quantifiable        | Phase 3 onward         |
| Consistency across assistants               | Qualitative                | Phase 4                |
| Demonstrable policy enforcement             | Qualitative; risk-reducing | Phase 1 onward         |
| Engineering authority retained by engineers | Qualitative                | Inherent to the design |

## 3. Market and User Analysis

### 3.1 Market Context

The addressable population comprises software engineering organizations that have adopted AI coding assistants and that maintain internal engineering standards they wish to see observed. This population is growing in proportion to assistant adoption, which is itself growing rapidly.

Three characteristics of the population are relevant to the design. Adoption of assistants is heterogeneous, so that a single organization commonly uses several different tools. Standards are organization-specific, so that a solution must be configurable rather than prescriptive. And engineering teams are resistant to tooling that impedes their workflow, so that latency and friction are adoption-critical rather than merely desirable properties.

### 3.2 Adjacent Categories

Several established categories of tooling address parts of the problem space. The table records their relationship to the proposed system without evaluative comment.

| Category                           | What It Provides                                                       | Relationship to This Project                                                      |
|------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Static analysis</b>             | Evaluation of code against general-purpose correctness and style rules | Complementary; does not evaluate organization-specific architectural intent       |
| <b>Testing frameworks</b>          | Verification of behaviour against expected outcomes                    | Complementary; verifies behaviour rather than structural conformance              |
| <b>Schema validation libraries</b> | Verification of data against a declared structure                      | Overlapping in part; operates on data rather than within the development workflow |
| <b>Output guardrail tools</b>      | Constraint of AI output, typically for general-purpose content         | Adjacent; oriented toward content safety rather than engineering standards        |
| <b>Observability platforms</b>     | Recording and analysis of system behaviour                             | Complementary; records occurrences without preventing them                        |

The position occupied by the proposed system is the combination of three properties that the adjacent categories do not jointly provide: deterministic enforcement rather than observation; evaluation against organization-specific architectural intent rather than general-purpose rules; and operation within the AI assistant's working loop rather than after it.

### 3.3 User Profiles

#### Profile A — Individual Developer

Works with an AI assistant daily. Values speed and flow, and will disable any tool that materially interrupts either. Adopts a governance tool only where it saves more time than it costs. The relevant benefit is immediate, specific feedback that shortens the corrective cycle.

#### Profile B — Technical Lead

Accountable for architectural integrity across a growing codebase and team. Presently relies on review, which is inconsistent and does not scale. The relevant benefit is automatic and uniform enforcement of architectural rules.

#### Profile C — Engineering Manager

Accountable for team output and its consistency. Observes divergence arising from heterogeneous tooling and an unpredictable review burden. The relevant benefit is a single standard applied to all work irrespective of the assistant used.

**Profile D — Compliance or Security Function**

Requires evidence that policy is enforced rather than assurance that it is stated. Cannot act upon probabilistic evidence. The relevant benefit is a deterministic, permanently recorded audit trail.

## 4. Product Requirements — Business Perspective

This section states requirements as business capabilities. Their technical expression is provided in the accompanying technical specification.

| ID    | Capability                                                      | Business Justification                                               | Priority    |
|-------|-----------------------------------------------------------------|----------------------------------------------------------------------|-------------|
| BR-01 | Enforce organization-specific standards on AI-generated output  | The primary requirement from which all others derive                 | Must have   |
| BR-02 | Produce identical verdicts for identical inputs                 | Without reproducibility the system cannot support policy or audit    | Must have   |
| BR-03 | Permit standards to be authored and controlled by engineers     | Retains engineering authority with the engineering team              | Must have   |
| BR-04 | Operate without perceptible impact on development velocity      | A tool that slows the workflow will be disabled by its users         | Must have   |
| BR-05 | Support graduated enforcement from observation to blocking      | Enables incremental adoption in an existing codebase                 | Must have   |
| BR-06 | Operate without dependency on external services                 | Supports restricted environments and removes an availability risk    | Must have   |
| BR-07 | Provide correction guidance sufficient for targeted remediation | Converts detection into resolution and reduces iteration cost        | Should have |
| BR-08 | Operate within the working loop of an AI coding assistant       | Prevention at the point of generation is superior to later detection | Should have |
| BR-09 | Preserve validation history durably                             | Enables continuity of work and audit evidence                        | Should have |
| BR-10 | Apply one standard to output from any assistant                 | Addresses divergence arising from heterogeneous tooling              | Should have |
| BR-11 | Integrate with version control and pipeline workflows           | Provides an enforcement point that cannot be bypassed                | Could have  |
| BR-12 | Surface recurring patterns of violation                         | Distinguishes systemic problems from isolated incidents              | Could have  |

## 5. Success Criteria and Measurement

### 5.1 Project Success Criteria

| ID    | Criterion                                            | Measure                                                                                                   |
|-------|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| SC-01 | All deliverables completed and accepted              | Every deliverable in Section 1.6 meets its acceptance criterion                                           |
| SC-02 | Objectives achieved within the planned schedule      | Each objective in Section 1.3 met within its target period                                                |
| SC-03 | System operates within its stated performance bounds | Measured latency within the configured allowance under representative load                                |
| SC-04 | Determinism demonstrated                             | Repeated execution over identical inputs produces identical verdicts                                      |
| SC-05 | Test coverage achieved                               | All components on the validation path covered by unit and adversarial tests                               |
| SC-06 | Documentation complete and coherent                  | A reader unfamiliar with the project can understand and integrate the system from the documentation alone |
| SC-07 | No unresolved high-severity defects                  | Defect register contains no unresolved item of high severity at closure                                   |

### 5.2 Product Success Indicators

The following indicators would demonstrate that the product achieves its intended effect. They are stated for completeness; their measurement extends beyond the present project period.

| Indicator                                 | What It Would Demonstrate                                                |
|-------------------------------------------|--------------------------------------------------------------------------|
| Reduction in corrective iterations        | That precise feedback shortens the path to an acceptable result          |
| Reduction in interaction expenditure      | That fewer and smaller interactions are required per task                |
| Violations prevented rather than detected | That enforcement operates at the point of generation                     |
| Reduction in time to resume work          | That preserved history reduces context reconstruction cost               |
| Consistency of outcome across assistants  | That the common standard achieves normalization                          |
| Continued voluntary use                   | That the tool provides more value than friction — the decisive indicator |

## 6. Risk Management

### 6.1 Method

Risks are assessed for probability and impact on a three-point scale, combined into a severity rating. Each risk is assigned a response strategy and an owner. The register is reviewed at each project milestone.

### 6.2 Risk Register

#### 6.2.1 Technical Risks

| ID   | Risk                                                                                   | Prob. | Impact | Sev. | Response                                                                                                    |
|------|----------------------------------------------------------------------------------------|-------|--------|------|-------------------------------------------------------------------------------------------------------------|
| R-01 | Validation latency exceeds the acceptable allowance, causing users to disable the tool | Low   | High   | Med  | Bounded execution with enforced time allowance; performance measured continuously from the outset           |
| R-02 | A defect in the system destabilizes a host application                                 | Low   | High   | Med  | Controlled failure design; adversarial testing; failure injection at every external boundary                |
| R-03 | Contract format proves insufficiently expressive for real requirements                 | Med   | Med    | Med  | Optional extended validation; contract format designed for extension                                        |
| R-04 | Contract authors rely on requirements the engine does not evaluate                     | Med   | High   | High | Explicit documentation of the evaluated set; diagnostic warning where an unevaluated requirement is present |
| R-05 | Concurrency defect produces intermittent incorrect behaviour                           | Med   | High   | High | Dedicated concurrency testing; conservative synchronization; independent state per tenant                   |
| R-06 | An interface relied upon for assistant integration changes                             | Med   | Med    | Med  | Integration isolated behind an abstraction; core unaffected by interface change                             |
| R-07 | A dependency introduces a security vulnerability                                       | Low   | High   | Med  | Minimized dependency surface; version pinning; automated vulnerability scanning                             |

#### 6.2.2 Project and Management Risks

| ID   | Risk                                                   | Prob. | Impact | Sev. | Response                                                                                             |
|------|--------------------------------------------------------|-------|--------|------|------------------------------------------------------------------------------------------------------|
| R-08 | Competing academic commitments reduce available effort | High  | Med    | High | Conservative estimates; strict prioritization; scope sequenced so an increment is always deliverable |
| R-09 | Scope expands beyond what the schedule permits         | High  | High   | High | Explicit exclusion list; change control; deferred capabilities recorded rather than built            |
| R-10 | Loss of project context between working sessions       | High  | Med    | High | Mandatory documentation of decisions; durable records maintained alongside code                      |
| R-11 | Uneven distribution of work across team members        | Med   | Med    | Med  | Defined roles; work allocation reviewed at each checkpoint                                           |

| ID   | Risk                                                      | Prob. | Impact | Sev. | Response                                                                            |
|------|-----------------------------------------------------------|-------|--------|------|-------------------------------------------------------------------------------------|
| R-12 | A team member becomes unavailable                         | Med   | High   | High | Documentation sufficient for another member to resume; no single point of knowledge |
| R-13 | Deliverables completed but presentation is inadequate     | Low   | Med    | Low  | Presentation rehearsed against the assessment criteria in advance                   |
| R-14 | Dependencies between phases are misjudged, causing rework | Med   | Med    | Med  | Dependency structure documented and reviewed before each phase commences            |

### 6.3 Principal Risks

Three risks carry the highest severity and warrant explicit attention.

- **R-04 — Reliance on unevaluated requirements.** This is the most consequential product risk because its failure mode is silent. A contract containing a requirement the engine does not evaluate reports conformance regardless, producing unwarranted confidence. Unwarranted confidence is more damaging than no control at all, because it displaces the vigilance that would otherwise apply. The response is treated as a safety measure rather than a documentation preference.
- **R-09 — Scope expansion.** The project addresses an inherently expansive problem space, and each capability suggests a further one. The response is an explicit exclusion list, recorded in Section 13, and a discipline of recording deferred capabilities rather than building them.
- **R-10 — Loss of context.** With part-time work distributed across a small team, intervals between working sessions are long and knowledge is readily lost. The response is that documentation is treated as a deliverable of equal standing with the software rather than as a subsequent activity.



## 7. Work Breakdown Structure

Work is decomposed to the level at which a package can be assigned to an individual and verified against a stated completion criterion.

| WBS        | Work Package                                  | Completion Criterion                                             |
|------------|-----------------------------------------------|------------------------------------------------------------------|
| <b>1.0</b> | <b>Project Initiation</b>                     |                                                                  |
| 1.1        | Team formation and role assignment            | Roles documented and agreed                                      |
| 1.2        | Problem analysis and requirements elicitation | Requirements recorded and reviewed                               |
| 1.3        | Project charter preparation                   | Charter approved by all parties                                  |
| 1.4        | Architectural design                          | Architecture documented; layer responsibilities defined          |
| <b>2.0</b> | <b>Core Engine Development</b>                |                                                                  |
| 2.1        | Configuration and error type foundation       | Configuration loaded and validated; error hierarchy defined      |
| 2.2        | Abstract interface definition                 | All required capabilities expressed as interfaces                |
| 2.3        | Rule engine implementation                    | All rules implemented; each covered by unit test                 |
| 2.4        | Validation orchestration                      | Complete validation path operational end to end                  |
| 2.5        | Contract loading and caching                  | Contracts loaded from local and remote sources; caching verified |
| <b>3.0</b> | <b>Supporting Infrastructure</b>              |                                                                  |
| 3.1        | Bounded execution runner                      | Time allowance and capacity limits demonstrably enforced         |
| 3.2        | Resilience controller                         | State progression verified by test                               |
| 3.3        | Event recording                               | Events emitted without blocking the validation path              |
| 3.4        | Structured diagnostics with redaction         | Sensitive fields redacted unconditionally                        |
| 3.5        | Composition root and lifecycle management     | System assembles and tears down deterministically                |
| <b>4.0</b> | <b>Quality Assurance</b>                      |                                                                  |
| 4.1        | Unit test suite                               | All components covered                                           |
| 4.2        | Adversarial test suite                        | Every documented limit exercised at its boundary                 |
| 4.3        | Concurrency and failure-injection testing     | Correct behaviour verified under both conditions                 |
| 4.4        | Static analysis and security scanning         | No unresolved findings                                           |
| <b>5.0</b> | <b>Integration</b>                            |                                                                  |
| 5.1        | Declarative integration mechanism             | Validation applied without alteration of function bodies         |
| 5.2        | Consultative service interface                | An assistant obtains and acts upon a structured verdict          |
| 5.3        | Pipeline and version control integration      | Verification stage operational                                   |
| <b>6.0</b> | <b>Documentation</b>                          |                                                                  |
| 6.1        | Concept brief                                 | Reviewed and approved                                            |
| 6.2        | Technical specification                       | Requirements traceable to architecture and verification          |
| 6.3        | Charter and management plan                   | Approved by stakeholders                                         |

| WBS        | Work Package                          | Completion Criterion                                    |
|------------|---------------------------------------|---------------------------------------------------------|
| 6.4        | Contract authoring guidance           | A newcomer can author a correct contract using it alone |
| 6.5        | Security assessment                   | All trust boundaries assessed                           |
| <b>7.0</b> | <b>Project Closure</b>                |                                                         |
| 7.1        | Final integration and verification    | All acceptance criteria satisfied                       |
| 7.2        | Presentation preparation and delivery | Delivered within the allotted time                      |
| 7.3        | Lessons learned                       | Recorded and reviewed by the team                       |

## 8. Project Schedule and Milestones

### 8.1 Milestones

| ID  | Milestone                | Criterion for Achievement                               | Target  |
|-----|--------------------------|---------------------------------------------------------|---------|
| M-1 | Project authorized       | Charter approved and signed                             | Week 2  |
| M-2 | Architecture established | Technical specification approved                        | Week 3  |
| M-3 | Core engine operational  | Validation path functional end to end                   | Week 7  |
| M-4 | Infrastructure complete  | All non-functional requirements demonstrably satisfied  | Week 9  |
| M-5 | Quality gate passed      | Test suite complete; no unresolved high-severity defect | Week 11 |
| M-6 | Integration demonstrated | An assistant obtains and applies a verdict              | Week 14 |
| M-7 | Documentation complete   | All documents approved                                  | Week 16 |
| M-8 | Project closed           | Presentation delivered; lessons recorded                | Week 18 |

### 8.2 Critical Path and Dependencies

The critical path runs through the core engine and its supporting infrastructure. The dependency structure is not a matter of preference but is imposed by the design: the enforcement foundation must exist before any capability that builds upon it, the durable record must exist and must have accumulated data before analysis of that data is meaningful, and consistency across assistants depends upon both.

| Activity                       | Depends Upon                                  | Nature of the Dependency                                         |
|--------------------------------|-----------------------------------------------|------------------------------------------------------------------|
| Rule engine                    | Interface definitions; value structures       | Cannot be constructed before its inputs are defined              |
| Validation orchestration       | Rule engine; execution runner; contract cache | Coordinates components that must exist first                     |
| Consultative service interface | Complete validation path                      | Exposes an operation that must already function                  |
| Durable history                | Event recording interface                     | Provides an alternative implementation of an existing capability |
| Historical analysis            | Durable history containing accumulated data   | Analysis of an empty record yields nothing                       |
| Multi-assistant consistency    | Validation core; historical analysis          | Requires both a common standard and observed performance data    |

Two consequences follow for the schedule. The core engine work cannot be compressed by parallelization, because its internal dependencies are sequential. And the later phases cannot be commenced early, because they depend upon data that earlier phases produce. Effort is therefore concentrated on the earlier phases, and the later phases are scheduled with the expectation that they may be curtailed if the schedule comes under pressure.

## 9. Team Structure and Responsibilities

### 9.1 Roles

| Role                              | Responsibilities                                                                                                 | Assigned To |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------|-------------|
| <b>Project Manager</b>            | Planning; work allocation; progress tracking; risk management; stakeholder communication; deliverable acceptance | _____       |
| <b>Technical Lead</b>             | Architectural decisions; technical standards; design review; resolution of technical impediments                 | _____       |
| <b>Developer (Core)</b>           | Implementation of the validation engine and domain logic; associated unit tests                                  | _____       |
| <b>Developer (Infrastructure)</b> | Implementation of supporting infrastructure; integration mechanisms                                              | _____       |
| <b>Quality Lead</b>               | Test strategy; adversarial and concurrency testing; defect triage; acceptance verification                       | _____       |
| <b>Documentation Lead</b>         | Documentation authorship and consistency; version control of documents; presentation preparation                 | _____       |

Where the team comprises fewer members than roles, roles are combined. The project manager and technical lead roles are held separately wherever team size permits, since the two apply different and occasionally competing forms of judgement.

### 9.2 Responsibility Assignment

R denotes Responsible; A denotes Accountable; C denotes Consulted; I denotes Informed.

| Activity                      | PM  | Tech Lead | Dev | QA  | Docs |
|-------------------------------|-----|-----------|-----|-----|------|
| Charter and planning          | A/R | C         | C   | C   | R    |
| Requirements definition       | A   | R         | C   | C   | R    |
| Architectural design          | I   | A/R       | C   | C   | I    |
| Core implementation           | I   | A         | R   | C   | I    |
| Infrastructure implementation | I   | A         | R   | C   | I    |
| Test strategy and execution   | I   | C         | C   | A/R | I    |
| Security assessment           | I   | A         | C   | R   | C    |
| Documentation                 | A   | C         | C   | C   | R    |
| Risk management               | A/R | C         | I   | C   | I    |
| Presentation                  | A   | C         | C   | C   | R    |

## 10. Communication Management Plan

### 10.1 Internal Communication

| Activity                       | Frequency                    | Participants                  | Purpose                                                          |
|--------------------------------|------------------------------|-------------------------------|------------------------------------------------------------------|
| <b>Progress checkpoint</b>     | Twice weekly                 | Full team                     | Progress against plan; impediments; next allocation              |
| <b>Technical review</b>        | Weekly                       | Technical lead; developers    | Design decisions; code review; resolution of technical questions |
| <b>Milestone review</b>        | At each milestone            | Full team                     | Verification of criteria; risk review; replanning if required    |
| <b>Written decision record</b> | On each significant decision | Decision maker; team informed | Preservation of rationale for later reference                    |

### 10.2 External Communication

| Audience                 | Mechanism                                          | Frequency                 | Content                                                      |
|--------------------------|----------------------------------------------------|---------------------------|--------------------------------------------------------------|
| <b>Course instructor</b> | Scheduled laboratory sessions; submitted documents | Per laboratory schedule   | Progress; deliverables; impediments requiring guidance       |
| <b>Peer teams</b>        | Shared reference material                          | As requested              | Documentation and approach shared for collaborative learning |
| <b>Class</b>             | Presentation                                       | At the designated session | Charter, approach and outcomes                               |

### 10.3 Documentation as Communication

The project treats written documentation as its principal communication mechanism rather than as a record produced after the fact. Two considerations motivate this. The team works part-time and asynchronously, so that intervals between working sessions are long and verbal context is not reliably retained. And a documented decision is durable in a way that a discussed decision is not: it survives the unavailability of the person who made it.

The practical rule adopted is that a decision is recorded at the time it is taken rather than reconstructed subsequently. Reconstruction is materially less accurate, and the rationale — which is the most valuable part of the record — is the part most readily lost.

## 11. Quality Management Plan

### 11.1 Quality Objectives

| Objective                    | Standard                                                                        | Verification                                           |
|------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------|
| <b>Correctness</b>           | The system produces the specified verdict for every specified input             | Unit and integration testing                           |
| <b>Reproducibility</b>       | Identical inputs produce identical verdicts on every execution                  | Repeated-execution testing                             |
| <b>Robustness</b>            | The system behaves correctly under hostile and degraded conditions              | Adversarial and failure-injection testing              |
| <b>Performance</b>           | Operation within the stated latency allowance                                   | Timed measurement under representative load            |
| <b>Security</b>              | All identified trust boundaries defended                                        | Threat assessment; security scanning; boundary testing |
| <b>Maintainability</b>       | Architectural invariants preserved                                              | Automated dependency analysis; static inspection       |
| <b>Documentation quality</b> | A newcomer can understand and integrate the system from the documentation alone | Review by a reader unfamiliar with the project         |

### 11.2 Quality Assurance Activities

- **Code review.** Every substantive change is reviewed by a team member other than its author before acceptance.
- **Automated verification.** Tests, linting, type checking and security scanning are executed on every change.
- **Architectural verification.** Structural invariants are verified by automated test rather than by convention, on the principle that a property enforced only by convention will eventually be violated.
- **Milestone review.** Deliverables are assessed against their acceptance criteria at each milestone rather than at project end.
- **Documentation review.** Each document is reviewed by a member other than its author for accuracy and comprehensibility.

### 11.3 Defect Management

| Severity        | Definition                                                                             | Response                                |
|-----------------|----------------------------------------------------------------------------------------|-----------------------------------------|
| <b>Critical</b> | Produces an incorrect verdict, breaches security, or destabilizes the host application | Work suspended until resolved           |
| <b>High</b>     | Materially impairs a required capability with no reasonable workaround                 | Resolved before the next milestone      |
| <b>Medium</b>   | Impairs a capability but a workaround exists                                           | Scheduled within the current phase      |
| <b>Low</b>      | Cosmetic or minor; no material effect on function                                      | Recorded; addressed as capacity permits |

## 12. Change and Scope Management

---

### 12.1 Rationale

Scope expansion is identified in Section 6 as one of the three principal risks. The problem domain is inherently expansive, and each capability delivered suggests further capabilities that appear both valuable and adjacent. Without an explicit control, the project would accumulate scope faster than it delivers.

### 12.2 Change Control Procedure

15. A proposed change is recorded in writing, with its justification and its estimated effect on scope, schedule and risk.
16. The project manager assesses the proposal against the objectives stated in Section 1.3.
17. A change that does not affect an objective or a deadline may be approved by the project manager within tolerance.
18. A change affecting an objective, a deadline or a deliverable requires the agreement of the full team.
19. An approved change is reflected in the plan, the schedule and the affected documentation before work commences.
20. A rejected change is recorded together with the reason for rejection, so that it is not reconsidered repeatedly.

### 12.3 Governing Principle

#### **The Discipline Applied**

A capability that is identified but not scheduled is recorded rather than built. The record ensures that the idea is not lost; the refusal to build it ensures that the schedule is not lost. Section 13 is the register of such capabilities, and its length is evidence of the discipline being applied rather than of opportunities being missed.

### 13. Exclusions and Deferred Scope

The following capabilities have been considered and deliberately excluded from the current scope. Each is recorded with the reason for its exclusion so that the decision is available for later reconsideration rather than repeatedly re-debated.

| Capability                                     | Reason for Exclusion                                                                                                          | Status                |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| <b>Graphical user interface</b>                | The system is a development-workflow component; its users work in editors and pipelines rather than in a separate application | Excluded              |
| <b>Centralized administrative service</b>      | Not required for the core function; local operation is sufficient for the intended scope                                      | Deferred              |
| <b>Distributed shared state</b>                | Unnecessary below a scale of multiple concurrent instances; adds substantial complexity for no present benefit                | Deferred              |
| <b>Model training or fine-tuning</b>           | Would introduce probabilistic behaviour into the component whose value depends on determinism                                 | Excluded on principle |
| <b>AI-generated correction guidance</b>        | Would introduce latency and non-reproducibility into the corrective path; a deterministic approach is preferred               | Excluded on principle |
| <b>Automatic contract inference</b>            | Contracts must express deliberate organizational intent; inferring them would defeat their purpose                            | Excluded on principle |
| <b>Semantic similarity search over history</b> | Structured queries satisfy the identified requirements; the additional complexity is not presently justified                  | Deferred              |
| <b>Role-based access control</b>               | Not required at the current deployment scale; relevant to a later multi-organization deployment                               | Deferred              |
| <b>Editor-integrated live feedback</b>         | Requires a separate protocol implementation of substantial size; the benefit does not presently justify the cost              | Deferred              |
| <b>Formal regulatory certification</b>         | Outside the scope of an academic project; would require external audit                                                        | Excluded              |

Three of these exclusions are marked as matters of principle rather than of sequencing. They concern the introduction of probabilistic behaviour into the system, and they are excluded permanently rather than deferred, because admitting them would compromise the property from which the system derives its value.



## 14. Conclusion

---

This document has established the formal charter authorizing the Congine project, the business case supporting it, and the management arrangements by which it will be delivered.

The project addresses a problem produced by a genuine and recent change in software engineering practice: the adoption of AI coding assistants has increased the rate at which software is produced without a corresponding increase in the assurance that what is produced conforms to the standards of the organization consuming it. The proposed response is a governance layer in which organizational requirements are expressed as contracts authored by engineers and enforced by deterministic rule-based evaluation.

The management approach reflects the constraints under which the project operates. A small team working part-time cannot deliver an expansive scope, and the plan therefore prioritizes strictly, sequences work according to genuine technical dependency rather than preference, and records deliberately excluded capabilities so that the exclusion is a decision rather than an oversight. Documentation is treated as a deliverable of equal standing with the software, on the assessment that in a project of this shape the loss of context between working sessions is a greater risk than any individual technical difficulty.

The team considers the selection of a genuine product, rather than a hypothetical scenario, to be an advantage for the purposes of this course. A real system imposes real dependencies that determine sequencing, real risks with real consequences, and real trade-offs between scope and schedule. These are precisely the conditions under which project management practice is meaningfully exercised rather than merely described.

### Charter Statement

The Congine project is hereby authorized to proceed as described in this charter. Its objective is to deliver a deterministic output governance engine that enforces human-authored engineering standards on AI-generated output, within the scope, schedule and constraints set out in Section 1, and under the management arrangements established in Sections 6 through 12.

*End of Document — CGN-DOC-03, Project Charter and Business Case, Version 1.0*